OpenCode.ai

The Open-Source AI Coding Agent for the Enterprise

Model-agnostic | Air-gap capable | Zero telemetry | MIT Licensed

March 2026

What Is OpenCode?

An **open-source AI coding agent** with 100K+ GitHub stars and 2.5M+ monthly developers.

> Unlike GitHub Copilot or Claude Code, OpenCode is **model-agnostic** -- it connects to 75+ LLM providers **and** local open-source models.

For enterprises, this means:

- Use your **best cloud model** (Claude Opus, GPT-5) for complex tasks
- Use **free local models** (Ollama, llama.cpp) for routine work
- **Keep all data inside your network** -- no vendor lock-in, no telemetry

The Cost Problem

Tool	Cost/Dev/Month	Data Leaves Org?
GitHub Copilot Business	$19	Yes -- sent to GitHub/Microsoft
GitHub Copilot Enterprise	$39	Yes -- sent to GitHub/Microsoft
Cursor Pro	$20-200	Yes -- sent to Cursor's servers
Claude Code (Max)	$100-200+	Yes -- sent to Anthropic

A 50-developer team pays $12,000-120,000/year on AI tools

...plus the hidden cost of sending all your proprietary code to third-party servers.

The OpenCode Model: Pay Only What You Use

Configuration	Cost/Dev/Month	When to Use
Local Models (Ollama)	**$0**	Routine coding, boilerplate, tests
Cloud API (Claude/GPT)	**$5-30**	Complex architecture, reasoning
GitHub Copilot sub	**$19**	Leverage existing Copilot investment
Mixed (local + cloud)	**$3-15**	80% local, 20% cloud

> OpenCode itself is **free**. You pay only raw API costs -- no markup, no per-seat licensing.

Mixed strategy = 60-80% cost reduction vs GitHub Copilot Enterprise.

50-Developer Team: Annual Cost

Approach	Annual Cost	Data Leaves Org?
GitHub Copilot Enterprise	**$23,400**	Yes
Cursor Business	**$48,000+**	Yes
OpenCode + Local Models	**$0-3,000**	**No**
OpenCode + Mixed	**$3,600-9,000**	Only cloud API calls

Model Flexibility: Use Any Model, Anytime

OpenCode Agent (Terminal / Desktop / IDE Extension)
connects via **AI SDK + Models.dev** to:

Cloud Providers	Local Models (Zero Cost)
Claude Opus / Sonnet (Anthropic)	**Ollama** / **llama.cpp** / **LM Studio**
GPT-5 (OpenAI)	**vLLM** / **Docker Model Runner**
Gemini (Google Vertex)	Qwen2.5-Coder, DeepSeek R1
GitHub Copilot (existing sub)	Llama 3.3, Phi-4

> **75+ providers supported.** Switch models in one config change -- no code changes needed.

Recommended Multi-Tier Strategy

Task	Model	Cost	Latency
Autocomplete, boilerplate, tests	**Qwen2.5-Coder 32B (local)**	$0	~200ms
Code review, refactoring	**Llama 3.3 70B (local)** / Copilot	$0-19/mo	~500ms
Complex architecture, debugging	**Claude Opus / GPT-5 (API)**	~$0.05-0.30/task	~2-5s
Multi-file agentic changes	**Claude Opus (API)**	~$0.50-2.00/task	~10-30s

> Use the cheapest model that gets the job done. Reserve cloud APIs for what local models can't handle.

The Agentic Layer

Copilot = Autocomplete. OpenCode = Agent.

Capability	GitHub Copilot	OpenCode
Inline autocomplete	Excellent	Use Copilot for this
Agentic multi-file edits	Limited	**Core strength**
Plan Mode	No	**Yes**
Tool use (MCP)	Limited	**Full MCP support**
Custom skills/agents	No	**Yes**
Terminal-native	No (IDE only)	**Yes**
Model choice	GitHub's only	**75+ providers + local**
Open source	No	**Yes (MIT)**

Best Strategy: Use Both Together

OpenCode and GitHub Copilot are **complementary, not competing**:

1. **GitHub Copilot** -- real-time inline autocomplete as you type
2. **OpenCode** -- agentic tasks: planning, multi-file changes, debugging, code review
3. **OpenCode uses your Copilot subscription** as one of its model providers

> Fast autocomplete + powerful agentic coding + local models for privacy.
> Best of all worlds.

Enterprise Setup: Zero Data Out

Everything stays inside your network

Developer Machines run **OpenCode** with `opencode.json`:

- `share: "disabled"` -- no data to opencode.ai
- `providers: internal only` -- whitelist your AI gateway
- `telemetry: off` -- zero instrumentation

Two paths for inference (both internal):

Path	Technology	Network Traffic
Internal AI Gateway	Azure OpenAI / Bedrock / vLLM	Internal only
Local Models	Ollama / llama.cpp on each machine	**Zero** -- fully air-gapped

SSO Provider integrates via central config.

> **Nothing leaves your network boundary.**

Enterprise Configuration Checklist

Setting	Configuration	Effect
Disable sharing	`"share": "disabled"`	No data sent to opencode.ai
Restrict providers	Whitelist internal gateway	All API calls through your infra
SSO integration	Central config + SSO	Your existing identity system
Disable external models	Block non-approved providers	Only approved models
Local-only mode	Ollama/llama.cpp	True air-gap, zero outbound
Private npm registry	`.npmrc` + Artifactory/Nexus	No public downloads
Central config	Single `opencode.json`	Consistent org-wide policy

Privacy and Telemetry: The Facts

Dimension	OpenCode	GitHub Copilot	Claude Code
Open source?	**Yes (MIT)**	No	No
Telemetry?	**None**	Yes (Microsoft)	Yes (Anthropic)
Code sent to vendor?	**No**	Yes	Yes
Fully air-gapped?	**Yes**	No	No
Config auditable?	**Yes** (JSON)	Partially	Partially

> OpenCode is the only tool where you can **audit every line of code**, run fully air-gapped, and guarantee zero data exfiltration.

OpenCode vs. Alternatives

Feature	OpenCode	Copilot	Claude Code	Cursor
License	MIT (free)	$19-39/mo	$100-200+/mo	$20-200/mo
Model lock-in	None (75+)	GitHub only	Anthropic	Limited
Local models	Full	No	No	Limited
Agentic coding	Full	Limited	Full	Moderate
MCP tools	Yes	Limited	Yes	Limited
Air-gapped	Yes	No	No	No
Data sovereignty	Complete	None	None	None
Custom agents	Yes	No	Limited	No

Use Case: The "AI Developer"

Regulatory Code Analysis at Scale

The Problem: Regulatory Code Analysis

In financial institutions, regulatory code analysis is **extremely labor-intensive**:

- **Attribute-level data lineage** for one calculation: **2-5 days** manually
- **Code verification** against ECB/EBA/IFRS documents: **3-7 days**
- **Critical data attribute mapping** across source systems: **2-4 days**
- Regulators (BCBS 239, ECB TRIM) demand **automated, auditable lineage**

> A single calculation (e.g., IFRS 9 LGD) involves **10-20 data attributes** flowing through SQL, Python, and Java across multiple systems.

The Solution: AI Developer Agent Pipeline

Inputs: Regulatory Document + Source Code + Data Source Definitions

Three agents run in sequence:

Step	Agent	What It Does	Output
1	**@lineage-analyst**	Traces every attribute: source → transforms → output	Lineage report with file:line refs
2	**@regulatory-verifier**	Maps each requirement to code	PASS / FAIL / PARTIAL matrix
3	**@cda-mapper**	Identifies Tier 1-4 critical attributes	CDA mapping + quality risks

> Each agent reads the previous agent's output for full context. The pipeline runs in **1-2 hours** vs **1-3 weeks** manually.

Three Specialized Agents

@lineage-analyst

Reads SQL, Python, PySpark, Java. Traces every attribute from **source system** through **transformations** to **output** -- with file:line references for every hop.

@regulatory-verifier

Takes a regulation document + code. Checks each requirement. Produces a **compliance matrix** with PASS / FAIL / PARTIAL / NOT FOUND status and severity ratings.

@cda-mapper

Identifies **critical data attributes** for a calculation. Maps each to source system, transformation chain, and output. Flags **data quality risks** and missing validations.

Worked Example: IFRS 9 LGD

Running all three agents against a Loss Given Default implementation:

Agent	Output	Key Findings
@lineage-analyst	8 attributes traced	5 complete, 2 hardcoded, 1 partial
@regulatory-verifier	8 requirements checked	3 PASS, 1 FAIL, 3 PARTIAL, 1 NOT FOUND
@cda-mapper	13 CDAs mapped	4 missing inputs, 1 fetched but unused

> **Critical finding**: Regulation requires downturn LGD linked to macro conditions. Code uses a **fixed 8% add-on** -- a material compliance gap that **manual review missed**.

ROI: AI Developer vs Manual Analysis

Metric	Manual	AI Developer	Improvement
Lineage per calculation	2-5 days	10-30 min	**40-100x faster**
Regulatory verification	3-7 days	15-45 min	**30-70x faster**
CDA mapping	2-4 days	10-30 min	**40-80x faster**
Full pipeline	**1-3 weeks**	**1-2 hours**	**40-60x faster**
Cost per analysis	$3,000-10,000	$0.50-5.00	**99%+ reduction**
Consistency	Analyst-dependent	Deterministic	Auditable

What's Included: Agent Templates

Complete project scaffold in `ai-developer-agents/`:

Component	Contents
3 Agent definitions	`lineage-analyst.md`, `regulatory-verifier.md`, `cda-mapper.md`
3 Skills	`/trace-lineage`, `/verify-regulation`, `/map-cda`
3 Output templates	Structured report templates
Worked example	IFRS 9 LGD: regulation + Python + SQL + data sources
Expected outputs	Complete sample reports
Enterprise config	`opencode.json` pre-configured for air-gap

Supported Regulatory Frameworks

Regulation	Issuer	Agent Use Cases
IFRS 9	IASB	ECL, PD, LGD, EAD lineage and verification
CRR/CRD (Basel III/IV)	ECB/EBA	RWA, capital ratio validation
BCBS 239	BCBS	Data lineage compliance, aggregation quality
FRTB	BCBS	Sensitivity calculation, bucket mapping
AnaCredit	ECB	Credit data attribute mapping
FINREP/COREP	EBA	Reporting attribute lineage

The Bottom Line

Why OpenCode for Your Enterprise

What	Why It Matters
60-80% cost reduction	Free tool + local models for routine, cloud only when needed
Zero vendor lock-in	Switch models in one config change
Complete data sovereignty	Open-source, no telemetry, air-gapped capable
Complementary to Copilot	Copilot for autocomplete + OpenCode for agentic tasks
Agentic > Autocomplete	Plan mode, MCP tools, multi-file edits, custom skills
Enterprise-ready	SSO, central config, provider whitelisting, share disabled

Recommended Next Step

> **Run a 2-week pilot with 5-10 developers.**

1. Install **OpenCode + Ollama** (Qwen2.5-Coder 32B) alongside existing GitHub Copilot
2. Measure: tasks completed, time saved, cost per developer, satisfaction
3. No procurement needed -- **it's free and open source**

```
npm install -g opencode
brew install ollama
ollama pull qwen2.5-coder:32b
opencode
```

Ready in 15 minutes. Zero cost. Zero risk.

Questions?

opencode.ai | github.com/opencode-ai/opencode

MIT Licensed | 100K+ GitHub Stars | 2.5M+ Monthly Developers

March 2026